SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Yoga Madha A	**Reg. No.:**	192425058
Section / Batch:	B	**Date:**	16-10-2026

Code of Conduct

I Yoga Madha A/192425058 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Yoga Madha A

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1.Problem Statement:
A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.
Develop a Python application that validates:
Email Address
Password
Mobile Number
using Regular Expressions.
Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password

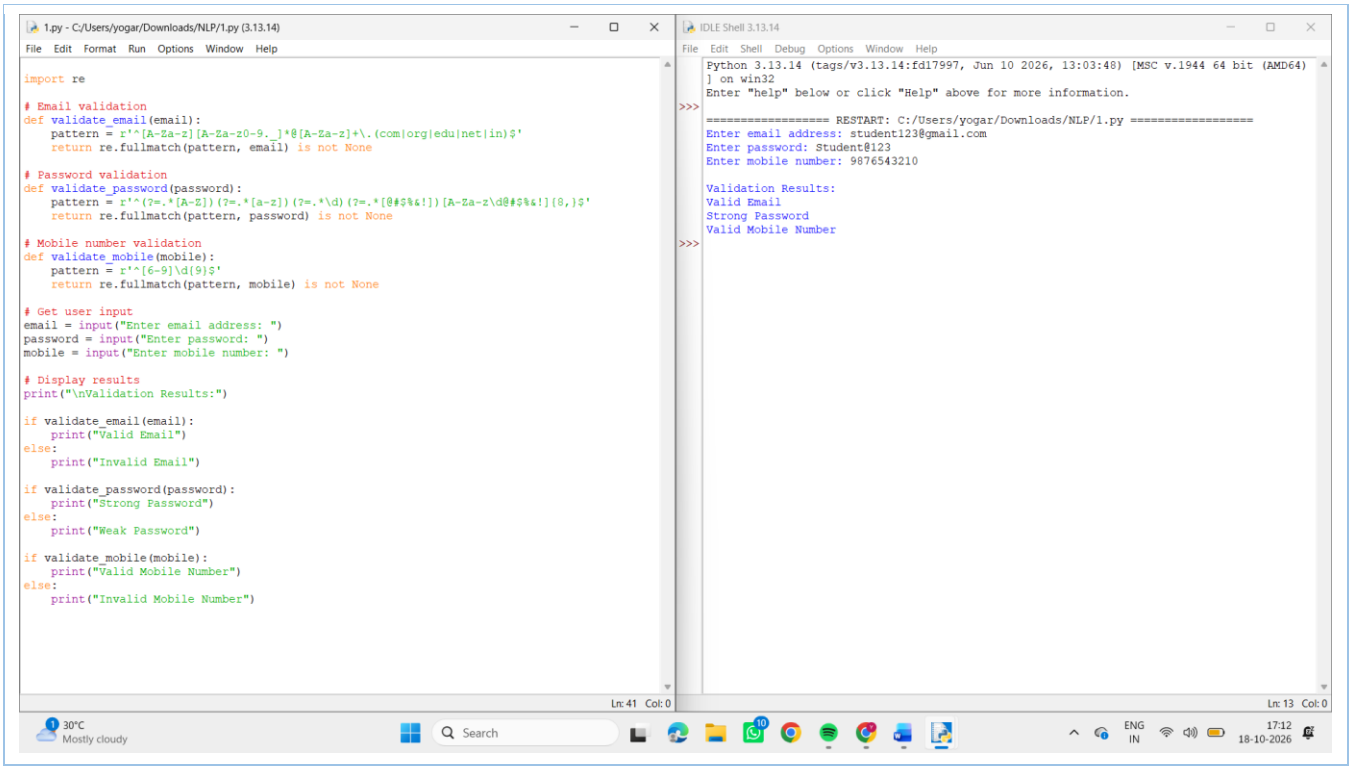
- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@,#,$,%,&,!)

Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output
Display:
Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Solution :
Aim
To develop a Python program using regular expressions to validate an email address, password, and mobile number according to predefined security rules.
Algorithm

1. Import the re module.
2. Get the email address, password, and mobile number from the user.
3. Define a regular expression for email validation.
4. Check whether the email starts with an alphabet and contains a valid domain and extension.
5. Define a regular expression for password validation.
6. Check for at least eight characters, one uppercase letter, one lowercase letter, one digit, and one allowed special character.
7. Define a regular expression for mobile number validation.
8. Check whether the mobile number contains exactly 10 digits and starts with a digit from 6 to 9.
9. Display the validation results.
10. End the program.

Code Implementation



Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement:
Develop a Python-based Deterministic Finite Automata (DFA) simulator.
The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example
Language:
Strings ending with "ab"

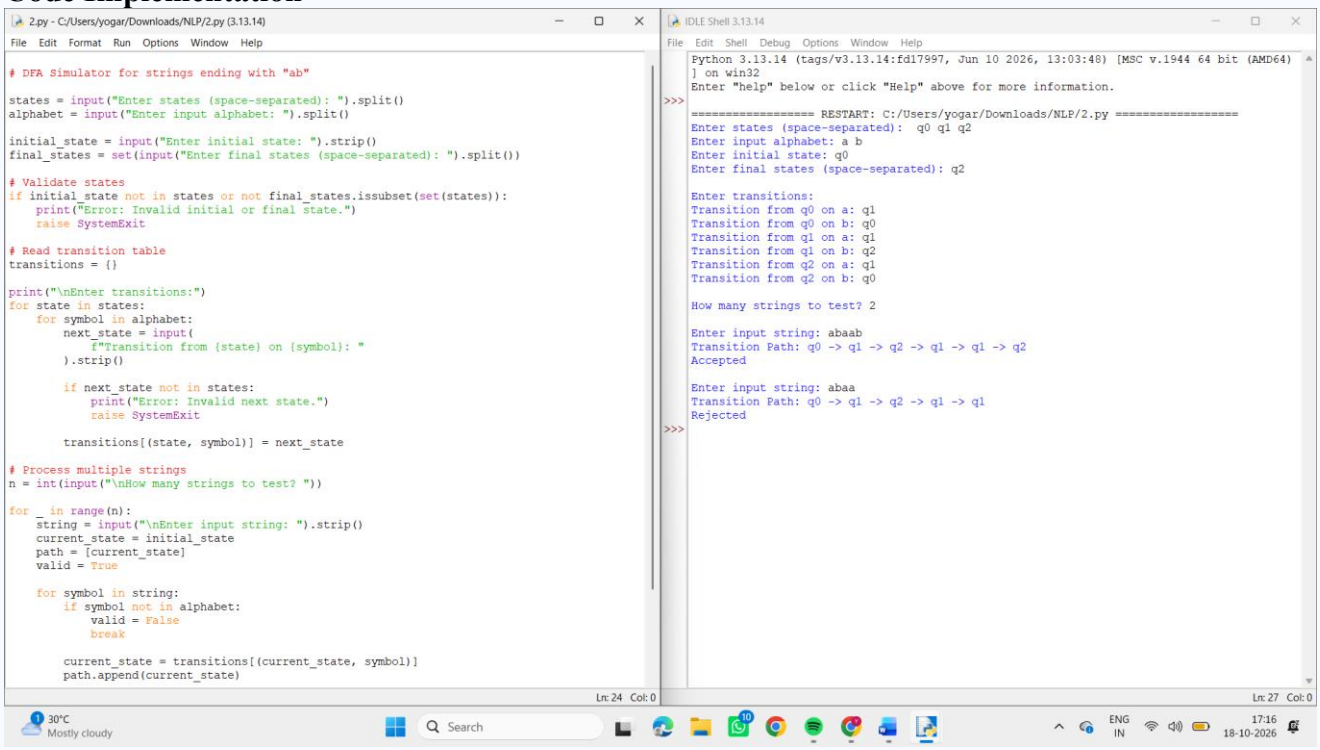
Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Solution :
Aim
To implement a Deterministic Finite Automaton (DFA) simulator in Python that accepts a DFA description, processes multiple input strings, and determines whether each string is accepted or rejected.
Algorithm

1. Get the states, input alphabet, initial state, and final states from the user.
2. Create a transition table using the current state and input symbol.
3. Validate the initial and final states.
4. Accept multiple input strings for testing.
5. Set the current state to the initial state for each input string.
6. Read the input string one character at a time.
7. Find the next state using the transition table.
8. Record and display the transition path.
9. If a transition is missing or an invalid symbol occurs, reject the string.
10. Accept the string if the final state belongs to the set of final states; otherwise, reject it.

Code Implementation



Section 3: Smart Pattern Matching Engine using Regular Expressions

3.Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

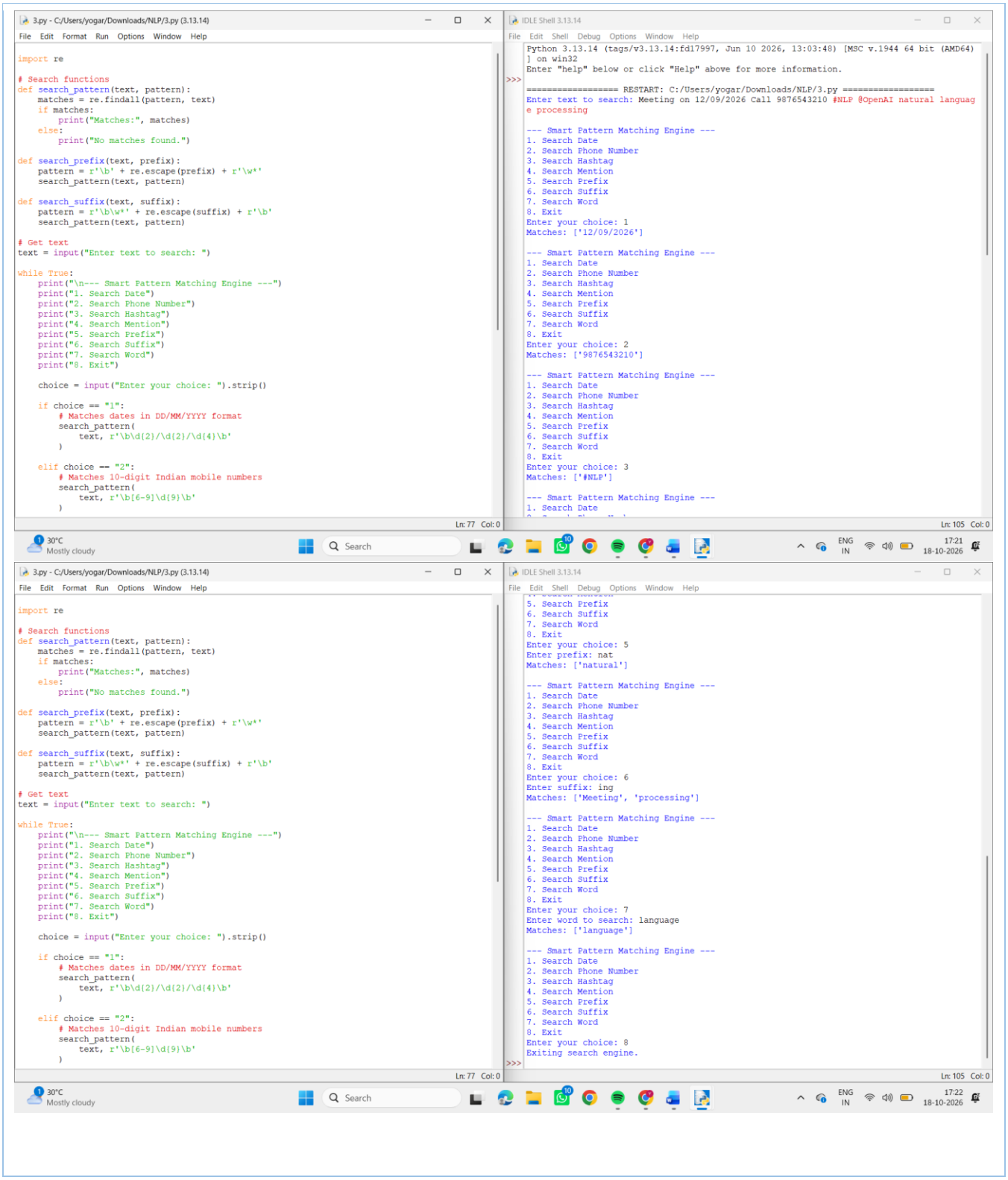
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Solution :
Aim
To develop a Python text search engine using regular expressions to search for dates, phone numbers, hashtags, mentions, prefixes, and suffixes in a given text.
Algorithm
1. Import the re module.
2. Accept a text paragraph from the user.
3. Display a menu of search operations.
4. Read the user's menu choice.
5. Define a regular expression for the selected pattern.
6. Use re.findall() to identify all matching patterns.
7. For prefix search, find words beginning with the specified prefix.
8. For suffix search, find words ending with the specified suffix.
9. Display all matches or a message if no matches are found.
10. Repeat the menu until the user chooses to exit.

Code Implementation



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints

		cases, and competitive requirements			
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____